INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
DE SANTA CATARINA

CÂMPUS SÃO JOSÉ

WAGNER FLORES DOS SANTOS

ANÁLISE COMPARATIVA ENTRE ARQUITETURAS DE AGENTE ÚNICO E MULTIAGENTE BASEADAS EM LLM PARA CHATBOTS DE ATENDIMENTO

SÃO JOSÉ

2026

Wagner Flores dos Santos

Análise Comparativa entre Arquiteturas de Agente Único e Multiagente Baseadas em LLM para Chatbots de Atendimento

Pré-projeto apresentado à Unidade Curricular de Trabalho de Conclusão de Curso I do Curso de Engenharia de Telecomunicações do Instituto Federal de Educação, Ciência e Tecnologia de Santa Catarina – Câmpus São José.

Área de concentração: Telecomunicações.

Orientador: Prof. Arliones Stevert Hoeller Júnior, Dr.

São José

2026

RESUMO

Os modelos de linguagem em grande escala (*Large Language Models* – LLMs) ampliaram as possibilidades de automação do atendimento ao cliente, permitindo interpretar solicitações, consultar informações e produzir respostas mais flexíveis do que as obtidas com fluxos baseados apenas em regras. Uma das abordagens exploradas consiste em distribuir as etapas do atendimento entre agentes especializados, coordenados para executar funções distintas. Ainda não está claro, porém, em quais condições essa distribuição de responsabilidades oferece vantagens suficientes para compensar o aumento da complexidade e do consumo de recursos em relação a uma arquitetura de agente único. Este trabalho tem como objetivo comparar experimentalmente as duas arquiteturas em um cenário controlado de atendimento textual de um provedor de Internet, considerando a correção e a relevância das respostas, a conclusão das tarefas, a precisão dos encaminhamentos, o número de chamadas ao modelo e o consumo de tokens. Para isso, serão implementados dois protótipos, submetidos ao mesmo conjunto de casos de teste e às mesmas condições de execução, com avaliação apoiada pelo DeepEval e por métricas registradas diretamente durante o experimento. Espera-se identificar em quais situações a especialização por agentes setoriais melhora o atendimento e quais efeitos essa organização produz sobre o número de chamadas ao modelo, o uso de tokens e o custo operacional.

Palavras-chave: modelos de linguagem em grande escala; sistemas multiagentes; chatbots; atendimento ao cliente; agentes inteligentes.

LISTA DE ILUSTRAÇÕES

Figura 1 – Organização conceitual das arquiteturas propostas 26

LISTA DE QUADROS

Quadro 1 – Síntese dos trabalhos relacionados 22
Quadro 2 – Correspondência entre objetivos específicos, etapas e entregas previstas 31

LISTA DE TABELAS

Tabela 1 – Distribuição prevista dos cenários conversacionais 29
Tabela 2 – Exemplo de cenário conversacional 30
Tabela 3 – Cronograma previsto para o TCC2 no primeiro semestre de 2027 . . . 36

LISTA DE ABREVIATURAS E SIGLAS

API *Application Programming Interface.*

JSON *JavaScript Object Notation.*

LLM *Large Language Model.*

SUMÁRIO

1 INTRODUÇÃO 9
1.1 PROBLEMA DE PESQUISA 10
1.2 OBJETIVOS 10
1.2.1 Objetivo geral 10
1.2.2 Objetivos específicos 10
1.3 DELIMITAÇÃO DO TRABALHO 11
1.4 ORGANIZAÇÃO DO TEXTO 12

2 REVISÃO BIBLIOGRÁFICA 13
2.1 FUNDAMENTAÇÃO TEÓRICA 13
2.1.1 Modelos de linguagem em grande escala 13
2.1.2 Agentes baseados em modelos de linguagem 14
2.1.3 Arquiteturas de agente único e multiagente 14
2.1.4 Comunicação, orquestração e retenção de contexto 16
2.1.5 Avaliação de sistemas conversacionais 17
2.2 TRABALHOS RELACIONADOS 17
2.3 SÍNTESE DA REVISÃO E LACUNA DE PESQUISA 21

3 METODOLOGIA 24
3.1 NATUREZA DO TRABALHO 24
3.2 DELINEAMENTO EXPERIMENTAL 24
3.2.1 Arquiteturas avaliadas 25
3.2.1.1 Arquitetura de agente único 25
3.2.1.2 Arquitetura multiagente 25
3.2.2 Variáveis controladas 26
3.2.3 Variáveis avaliadas 27
3.3 BASE DE CONHECIMENTO E PROMPTS 28
3.4 CENÁRIOS CONVERSACIONAIS 28
3.4.1 Cenário conversacional de exemplo 29
3.5 MATERIAIS E FERRAMENTAS 30
3.6 PROCEDIMENTO EXPERIMENTAL 31
3.7 MÉTRICAS E CRITÉRIOS DE AVALIAÇÃO 32
3.8 PRIVACIDADE E PROTEÇÃO DOS DADOS 34
3.9 RISCOS E LIMITAÇÕES 34

4 CRONOGRAMA DE ATIVIDADES 36

Referências **37**

1 INTRODUÇÃO

Os *Large Language Model*s (LLMs) ampliaram as possibilidades de automação de atividades que envolvem compreensão e geração de linguagem natural. Quando integrados a bases de conhecimento, ferramentas e mecanismos de decisão, esses modelos podem formar agentes capazes de interpretar solicitações, manter o contexto de uma conversa e executar tarefas com algum grau de autonomia. No atendimento ao cliente, essa abordagem permite desenvolver soluções mais flexíveis do que os chatbots baseados apenas em regras e fluxos predefinidos.

Trabalhos recentes apresentam aplicações de LLMs na classificação de chamados e na geração de respostas de suporte (Moncada *et al.*, 2025), bem como arquiteturas formadas por agentes coordenadores e especializados para atuar em diferentes componentes de sistemas de telecomunicações (Lee; Tseng; Chen, 2025). Essas soluções podem ser organizadas em arquiteturas de agente único ou multiagente.

Na arquitetura de agente único, um mesmo componente interpreta as mensagens, consulta as informações disponíveis, decide o procedimento e produz as respostas. Essa centralização reduz a necessidade de coordenação entre componentes. Na arquitetura multiagente, essas responsabilidades são distribuídas entre agentes com funções específicas, como triagem, primeiro atendimento, comercial, ouvidoria e suporte técnico. A divisão pode facilitar a especialização e reduzir o contexto recebido por cada agente, mas também introduz etapas adicionais de comunicação e novos pontos de falha (Yan *et al.*, 2026).

A comunicação entre os agentes influencia a eficiência, a escalabilidade e a robustez do sistema. Uma decisão incorreta na triagem pode encaminhar a conversa para um agente que não possui as informações necessárias. Também podem ocorrer perdas de contexto, encaminhamentos desnecessários ou propagação de erros entre os componentes. Falhas de agentes e interrupções na comunicação também são apontadas como riscos desses sistemas (Owotogbe, 2025).

Estudos recentes investigam o encaminhamento de conversas para atendimento humano e o reconhecimento de intenções em ambientes multiagentes (Amri *et al.*, 2026; Yang; Xu; Zhang, 2025). Os resultados apontam benefícios da especialização, mas também mostram que a vantagem da arquitetura multiagente não ocorre em todas as métricas. Além disso, os trabalhos adotam diferentes modelos, cenários e critérios de avaliação, o que dificulta uma comparação direta entre arquiteturas de agente único e multiagente. Na comparação realizada por Amri et al. (Amri *et al.*, 2026), também foi observada uma tendência de o sistema multiagente realizar transferências para atendimento humano mais cedo.

Esses resultados indicam que a adoção de uma arquitetura multiagente não representa, necessariamente, uma melhoria em todas as situações. Ainda são necessárias comparações nas quais as arquiteturas utilizem o mesmo modelo, a mesma base de conhecimento, parâmetros equivalentes e cenários de atendimento comparáveis. Também é necessário observar o desenvolvimento da conversa, as decisões de encaminhamento e os recursos utilizados até a conclusão da tarefa.

1.1 PROBLEMA DE PESQUISA

A escolha entre uma arquitetura de agente único e uma arquitetura multiagente envolve um compromisso entre qualidade, complexidade e custo operacional. A especialização pode melhorar o tratamento de determinadas solicitações, mas exige coordenação entre agentes e pode aumentar o número de chamadas ao modelo e o consumo de tokens.

Uma avaliação baseada somente em perguntas isoladas não representa todo o funcionamento de um chatbot de atendimento. Em uma conversa real, as informações são fornecidas gradualmente, e uma resposta pode depender das mensagens anteriores. O sistema também pode precisar rever o encaminhamento quando o usuário apresenta uma nova informação ou muda de assunto durante o atendimento.

Diante desse contexto, este trabalho busca responder à seguinte pergunta:

> Quais diferenças de desempenho são observadas entre uma arquitetura de agente único e uma arquitetura multiagente, ambas baseadas em modelos de linguagem em grande escala e aplicadas a cenários conversacionais de atendimento, quanto à correção e à relevância das respostas, à conclusão das tarefas, à precisão dos encaminhamentos, ao número de chamadas ao modelo e ao consumo de tokens?

1.2 OBJETIVOS

1.2.1 Objetivo geral

Comparar experimentalmente o desempenho de uma arquitetura de agente único e de uma arquitetura multiagente, ambas baseadas em modelos de linguagem em grande escala e aplicadas a cenários conversacionais de atendimento textual de um provedor de Internet.

1.2.2 Objetivos específicos

Para alcançar o objetivo geral, foram definidos os seguintes objetivos específicos:

- definir uma base de conhecimento e regras de atendimento comuns às duas arquiteturas;
- especificar e implementar um protótipo de agente único e um protótipo multiagente composto por um agente de triagem e agentes especializados;
- elaborar e revisar um conjunto de cenários conversacionais de teste para primeiro atendimento, comercial, ouvidoria e suporte técnico;
- avaliar o desempenho das duas arquiteturas quanto à correção e à relevância das respostas, à conclusão das tarefas, à precisão dos encaminhamentos, ao consumo de tokens, ao número de chamadas ao modelo e ao custo estimado de execução;
- comparar os resultados para identificar as vantagens, limitações e custos de cada arquitetura.

1.3 DELIMITAÇÃO DO TRABALHO

O estudo será restrito ao atendimento textual e utilizará cenários sintéticos baseados em situações típicas de atendimento de um provedor de Internet. Todos os dados empregados serão fictícios, sem utilização de conversas ou cadastros de clientes reais e sem interação com clientes durante o experimento.

As duas arquiteturas utilizarão o mesmo modelo de linguagem, os mesmos parâmetros de geração e a mesma base de conhecimento. No agente único, as regras e informações serão reunidas em um único contexto. No sistema multiagente, o conteúdo será distribuído conforme as responsabilidades da triagem e dos agentes especializados.

Serão consideradas quatro áreas de atendimento: primeiro atendimento, comercial, ouvidoria e suporte técnico. Os cenários poderão conter mudanças de assunto durante a conversa, permitindo verificar o comportamento das arquiteturas diante da necessidade de um novo encaminhamento.

Não fazem parte do escopo o atendimento por voz, o processamento de conteúdo multimodal, o treinamento ou ajuste fino de modelos, a comparação entre diferentes LLMs e a implantação do sistema com clientes reais. Também não serão realizados testes de robustez com injeção deliberada de falhas nos agentes ou na comunicação.

A latência das respostas não será avaliada. Como o acesso ao modelo será realizado por uma API externa, o tempo observado poderá variar conforme as condições da rede e do serviço, sobre as quais o experimento não terá controle. A comparação dos recursos utilizados ficará restrita ao número de chamadas ao modelo, ao consumo de tokens e ao custo estimado de execução.

1.4 ORGANIZAÇÃO DO TEXTO

Além deste capítulo introdutório, a proposta está organizada da seguinte forma: o Capítulo 2 apresenta a fundamentação teórica, os trabalhos relacionados e a lacuna de pesquisa; o Capítulo 3 descreve as arquiteturas propostas, a base de conhecimento, os cenários conversacionais, as ferramentas, o procedimento experimental e os critérios de avaliação; e o Capítulo 4 apresenta o cronograma das atividades previstas para o desenvolvimento do trabalho no TCC2.

2 REVISÃO BIBLIOGRÁFICA

Este capítulo apresenta os conceitos utilizados no desenvolvimento do trabalho e discute pesquisas relacionadas ao uso de modelos de linguagem em sistemas de atendimento. São abordados os agentes baseados em modelos de linguagem, as arquiteturas de agente único e multiagente, a comunicação entre agentes e os critérios empregados na avaliação de sistemas conversacionais.

2.1 FUNDAMENTAÇÃO TEÓRICA

2.1.1 Modelos de linguagem em grande escala

Os modelos de linguagem em grande escala são capazes de interpretar instruções e produzir textos a partir do contexto fornecido em sua entrada. Em sistemas de atendimento, esses modelos podem ser utilizados para identificar a intenção do usuário, consultar informações e elaborar respostas adequadas à solicitação recebida.

Diferentemente de um fluxo baseado somente em regras fixas, um sistema apoiado em LLMs pode tratar diferentes formas de expressar uma mesma intenção. Essa característica é útil em conversas nas quais o usuário não segue um roteiro predefinido ou fornece as informações de maneira incompleta.

O comportamento do modelo depende das instruções, das informações disponíveis e do histórico incluído no contexto. Uma resposta também pode variar entre execuções, mesmo quando a entrada é semelhante. Por esse motivo, a utilização de LLMs em processos de atendimento exige critérios que permitam verificar a correção das respostas e o cumprimento das regras definidas para o serviço.

As informações processadas pelo modelo são representadas por tokens, que correspondem a unidades de texto utilizadas na entrada e na saída. A quantidade de tokens depende do tamanho dos prompts, do histórico da conversa, da base de conhecimento fornecida e das respostas produzidas. Como arquiteturas multiagentes podem realizar mais de uma chamada ao modelo para tratar uma mesma mensagem, o número de chamadas e o consumo de tokens são medidas relevantes para a comparação proposta.

Yang, Xu e Zhang observam que modelos de linguagem podem melhorar o reconhecimento de intenções em comparação com métodos baseados somente em regras ou classificação tradicional. Os autores também destacam o custo computacional e a latência associados ao seu uso (Yang; Xu; Zhang, 2025). Embora a latência não seja uma variável avaliada neste trabalho, o estudo demonstra que a utilização de LLMs acrescenta custos operacionais que devem ser considerados na escolha da arquitetura.

2.1.2 Agentes baseados em modelos de linguagem

Um agente baseado em modelo de linguagem combina o modelo com instruções que definem sua função, informações de contexto e mecanismos para executar ações. Em um chatbot de atendimento, essas ações podem incluir classificar uma solicitação, consultar uma base de informações, formular uma resposta ou encaminhar a conversa para outro setor.

As instruções fornecidas ao agente delimitam sua atuação. Um agente de suporte técnico, por exemplo, pode receber regras sobre os testes permitidos, os dados que devem ser solicitados e as situações que exigem transferência para atendimento humano. O histórico da conversa permite utilizar informações fornecidas em mensagens anteriores e evitar que o usuário tenha que repeti-las.

Agentes baseados em modelos de linguagem também podem utilizar mecanismos de memória e recuperação de informações. Yan *et al.* (2026) destacam que técnicas de geração aumentada por recuperação podem fornecer ao modelo informações obtidas de fontes externas ou de memórias estruturadas. Nesse tipo de abordagem, os conteúdos relevantes são recuperados dinamicamente e inseridos no contexto utilizado para produzir a resposta.

Neste trabalho não será implementado um mecanismo de geração aumentada por recuperação. A base de conhecimento será formada por documentos previamente definidos, contendo regras, informações e procedimentos fictícios de atendimento. No agente único, todo o conteúdo será disponibilizado em um contexto comum. Na arquitetura multiagente, o mesmo conteúdo será dividido conforme a responsabilidade de cada agente.

Moncada et al. utilizam agentes especializados para realizar a ingestão de dados, a classificação de chamados e a geração de respostas em uma aplicação de suporte ao cliente (Moncada *et al.*, 2025). O trabalho mostra que uma tarefa de atendimento pode ser dividida entre componentes com responsabilidades distintas. Essa divisão, entretanto, exige que as informações necessárias sejam transmitidas corretamente entre os agentes.

2.1.3 Arquiteturas de agente único e multiagente

Na arquitetura de agente único, um mesmo agente concentra as funções necessárias ao atendimento. Ele recebe as mensagens, interpreta a solicitação, mantém o histórico, consulta as informações disponíveis e produz a resposta. As regras dos diferentes setores ficam reunidas no contexto desse agente, o que reduz a necessidade de encaminhamentos internos. Por outro lado, o aumento da quantidade de regras e assuntos pode tornar o contexto mais extenso e dificultar a separação das responsabilidades, exigindo que o agente identifique quais instruções devem ser aplicadas em cada situação.

Na arquitetura multiagente, as responsabilidades são divididas entre agentes com

funções específicas. Lee, Tseng e Chen (Lee; Tseng; Chen, 2025) propõem essa organização para sistemas de suporte à operação de telecomunicações, distinguindo um agente principal (*Chief Agent*) e agentes especializados (*Specific Agents*). O agente principal utiliza LangGraph para organizar o fluxo de execução e adota o paradigma ReWOO, que separa o planejamento das ações, a execução das ferramentas e a elaboração da resposta final. Os agentes especializados representam funções de diferentes sistemas e acessam informações por meio de ferramentas, APIs e extração de dados de páginas web. A proposta ilustra a separação entre coordenação e execução de tarefas específicas. Entretanto, os benefícios de eficiência, escalabilidade e flexibilidade discutidos pelos autores não são acompanhados de uma avaliação quantitativa própria que compare as arquiteturas de agente único e multiagente sob condições equivalentes.

Amri et al. (Amri *et al.*, 2026) comparam arquiteturas de agente único e multiagente na decisão de encaminhar conversas para atendimento humano. No agente único, um mesmo agente analisa a solicitação e aciona ferramentas de recuperação de informações ou transferência. Na arquitetura multiagente, o processamento inclui a coleta de contexto, a extração de dados do cliente em conversas em andamento e um agente de roteamento, que direciona a solicitação a agentes de perguntas frequentes, reclamações ou transferência. Os experimentos utilizam os modelos GPT-4.1 e Gemini 2.5 Flash sobre o conjunto QiscusCS, composto por 100 diálogos e 4.265 enunciados. Com GPT-4.1, o Macro F1 aumenta de 0,5356 no agente único para 0,6159 no multiagente; com Gemini 2.5 Flash, passa de 0,3333 para 0,6260. Entretanto, a AUC com GPT-4.1 diminui de 0,7017 para 0,6881, enquanto com Gemini aumenta de 0,6053 para 0,6646. Esses resultados mostram que os ganhos dependem do modelo e da métrica considerada, reforçando a necessidade de comparar as arquiteturas sob condições equivalentes.

Yan *et al.* (2026) classificam as arquiteturas de comunicação de sistemas multiagentes baseados em modelos de linguagem em estruturas planas, hierárquicas, por equipes, de sociedade e híbridas. Nas estruturas hierárquicas, componentes de nível superior coordenam ou direcionam a execução de agentes responsáveis por tarefas específicas. Essa organização facilita a divisão das responsabilidades, mas acrescenta etapas de comunicação e pode concentrar decisões em determinados componentes.

A arquitetura multiagente proposta neste trabalho aproxima-se de uma organização hierárquica com coordenação centralizada. Um orquestrador será responsável por manter o estado da conversa e controlar a execução. O agente de triagem selecionará o primeiro agente especializado. Enquanto o assunto permanecer na mesma área, as mensagens serão encaminhadas ao agente ativo. Quando esse agente identificar uma mudança de assunto, solicitará uma nova triagem, sem escolher diretamente o próximo especialista.

2.1.4 Comunicação, orquestração e retenção de contexto

A orquestração corresponde ao controle do fluxo de execução entre os componentes do sistema. Em uma arquitetura multiagente de atendimento, o orquestrador recebe a mensagem, recupera o estado da conversa e aciona a triagem ou o agente especializado que se encontra ativo. O orquestrador não produz a resposta de atendimento, mas controla quais componentes serão executados e registra as decisões tomadas.

A triagem é responsável por classificar o assunto e indicar o agente adequado. Depois dessa decisão, o agente especializado permanece ativo enquanto as mensagens continuarem relacionadas à sua área. Caso identifique uma mudança de assunto, ele retorna ao orquestrador uma solicitação de nova triagem. Dessa forma, os especialistas não realizam transferências diretas entre si.

Em uma conversa com vários turnos, as informações costumam ser fornecidas gradualmente. O usuário pode informar inicialmente que está sem acesso à Internet e somente depois relatar o estado das luzes do equipamento. Se houver troca de agente entre essas mensagens, o novo agente precisa receber as informações necessárias para continuar o atendimento.

Para representar esse comportamento, o sistema deverá manter o histórico das mensagens, os fatos relevantes extraídos da conversa e a identificação do agente ativo. O histórico preservará as mensagens trocadas; os fatos relevantes poderão registrar dados como CEP, condição de cliente e problema relatado; e a identificação do agente ativo indicará qual componente deve receber a mensagem seguinte.

A comunicação entre os agentes não ocorrerá de maneira livre. As trocas serão intermediadas pelo orquestrador e utilizarão saídas estruturadas, contendo a intenção identificada, o destino indicado, a ação solicitada e os fatos que devem permanecer no contexto. Essa estrutura permitirá registrar o caminho percorrido por cada conversa e verificar em qual etapa ocorreu um possível erro.

Yan *et al.* (2026) analisam os sistemas multiagentes a partir da comunicação estabelecida entre seus componentes. Segundo essa perspectiva, não basta avaliar separadamente a capacidade de cada agente, pois o resultado também depende das mensagens trocadas, do conteúdo transferido e das decisões tomadas durante a execução.

Falhas em um componente podem afetar as etapas seguintes de um sistema multiagente. Owotogbe (2025) discute esse problema ao investigar a robustez de sistemas formados por agentes baseados em modelos de linguagem. Embora a injeção deliberada de falhas não faça parte do escopo deste trabalho, a possibilidade de propagação de erros justifica o registro do fluxo percorrido por cada atendimento.

2.1.5 Avaliação de sistemas conversacionais

A avaliação de um sistema conversacional pode considerar tanto a resposta entregue ao usuário quanto o processo utilizado para produzi-la. Uma resposta pode estar correta, mas ter sido obtida após encaminhamentos desnecessários, repetição de perguntas ou uso excessivo de chamadas ao modelo.

A correção indica se a resposta respeita as informações e regras disponíveis. A relevância verifica se o conteúdo produzido trata diretamente da solicitação apresentada. A retenção de conhecimento observa se o sistema preserva informações fornecidas em turnos anteriores. A conclusão da tarefa verifica se as necessidades definidas para o cenário foram atendidas.

Neste trabalho, parte da avaliação será realizada com o DeepEval, uma biblioteca destinada à avaliação de aplicações baseadas em modelos de linguagem (Confident AI, 2026). As mensagens do usuário e do assistente serão organizadas como casos de teste conversacionais, aos quais serão aplicadas as métricas inicialmente previstas de relevância dos turnos, retenção de conhecimento e completude da conversa. Essas métricas utilizam um modelo de linguagem como avaliador e fornecem pontuações e justificativas para analisar, respectivamente, a pertinência das respostas ao contexto, a preservação das informações e o atendimento das intenções apresentadas durante a conversa.

Além da avaliação, o DeepEval será utilizado como apoio à preparação dos cenários sintéticos. As descrições e os desfechos sugeridos pela ferramenta serão revisados pelo pesquisador antes da elaboração dos roteiros definitivos, conforme o procedimento apresentado na seção de cenários conversacionais.

Informações objetivas, como a fila selecionada, o número de chamadas ao modelo, o consumo de tokens e a ocorrência de uma transferência, serão verificadas diretamente nos registros da execução. Essa combinação permite complementar a avaliação por LLM com medidas obtidas sem o julgamento do modelo.

Como a avaliação baseada em modelo pode apresentar variações ou interpretações inadequadas, as duas arquiteturas serão analisadas com o mesmo modelo avaliador, as mesmas configurações e os mesmos critérios. Amostras das avaliações automáticas também serão conferidas manualmente.

2.2 TRABALHOS RELACIONADOS

Os trabalhos analisados abordam diferentes aspectos do uso de modelos de linguagem e sistemas multiagentes. Alguns apresentam aplicações voltadas ao atendimento, enquanto outros tratam do reconhecimento de intenções, da comunicação entre agentes e da robustez dessas arquiteturas.

Moncada et al. (Moncada *et al.*, 2025) apresentam uma arquitetura multiagente para automatizar a classificação e a geração de respostas a chamados de suporte técnico. A solução utiliza o framework CrewAI e o modelo GPT-4.1-mini, distribuindo o processamento entre três agentes: o *Reader Agent*, responsável pela leitura e estruturação dos dados de um arquivo CSV; o *Classifier Agent*, que classifica os chamados como incidente, problema, solicitação ou mudança; e o *Analyzer Agent*, que analisa a prioridade, extrai palavras-chave e produz uma resposta contextualizada.

A avaliação utiliza uma amostra de 20 chamados do conjunto *Customer IT Support Ticket*. O classificador acerta 17 casos, alcançando acurácia de 85%. Entretanto, o desempenho varia entre as categorias: o F1 é de 1,00 para solicitações e mudanças, 0,77 para incidentes e 0,40 para problemas. Os autores associam os erros principalmente à dificuldade de distinguir incidentes de problemas em descrições ambíguas. Para avaliar as respostas, utilizam representações vetoriais produzidas pelo modelo *all-MiniLM-L6-v2* e calculam a similaridade de cosseno com as respostas de referência, obtendo média de 0,756.

O estudo exemplifica a divisão do processamento de chamados entre agentes especializados e combina a avaliação da classificação com a análise semântica das respostas. Contudo, a amostra reduzida limita a abrangência dos resultados, e a similaridade com uma resposta de referência não demonstra, isoladamente, a resolução correta da solicitação. Além disso, o trabalho não apresenta uma comparação controlada com uma arquitetura de agente único nem avalia a continuidade do atendimento em conversas com múltiplos turnos e mudanças de assunto, aspectos contemplados na proposta deste TCC.

No contexto das telecomunicações, Lee, Tseng e Chen (2025) propõem uma arquitetura multiagente para integrar sistemas de suporte à operação por meio de interações em linguagem natural. A organização segue princípios de projeto orientado a domínio, dividindo as responsabilidades entre um agente principal (*Chief Agent*) e agentes especializados (*Specific Agents*), associados a funções dos sistemas operacionais. O agente principal utiliza LangGraph para coordenar o fluxo e adapta o paradigma ReWOO às etapas de planejamento, execução e consolidação dos resultados. Os agentes especializados acessam dados e executam tarefas por meio de ferramentas, extração de informações de páginas web e APIs. A integração com LangServe permite disponibilizar os agentes como serviços acessíveis por interfaces REST.

Os autores discutem benefícios de modularidade, flexibilidade na escolha dos modelos e possibilidade de execução paralela. Entretanto, não apresentam uma avaliação quantitativa própria que permita medir esses benefícios em comparação com uma arquitetura de agente único. Os ganhos de eficiência no uso de tokens e de acurácia mencionados na descrição do ReWOO são atribuídos ao trabalho de referência desse paradigma, não a um experimento conduzido sobre a arquitetura de telecomunicações proposta. Assim,

o estudo contribui como referência de organização e integração entre agentes, mas não permite concluir em quais condições essa organização supera uma solução centralizada.

O reconhecimento de intenções é investigado por Yang, Xu e Zhang (2025), que propõem uma abordagem em duas etapas para selecionar agentes em uma plataforma de aplicações de inteligência artificial. Na primeira etapa, o sistema utiliza TF-IDF (*Term Frequency–Inverse Document Frequency*), uma técnica que representa textos por pesos atribuídos às palavras, considerando sua frequência no texto e sua raridade no conjunto de documentos. O sistema calcula a similaridade de cosseno entre essas representações da consulta do usuário e dos nomes e descrições dos agentes disponíveis. Quando a similaridade supera o limiar de 0,1, o agente correspondente é selecionado. Caso contrário, o sistema aciona um LLM para interpretar a solicitação considerando o texto de entrada e os metadados dos agentes.

A avaliação utiliza 30 consultas sintéticas geradas com o modelo Qwen2.5-Turbo, também empregado nos experimentos de reconhecimento de intenções. Os autores comparam três configurações: TF-IDF executado no cliente, TF-IDF executado na nuvem e reconhecimento diretamente por LLM. As duas configurações de TF-IDF apresentam acurácia de 43,5%, com tempos médios de resposta de 1,28 ms no cliente e 14,56 ms na nuvem. O LLM alcança acurácia de 77,2%, mas apresenta tempo médio de 2.005,94 ms. Os resultados evidenciam um compromisso entre acurácia e tempo de resposta, motivando o uso do método mais leve antes do acionamento do modelo de linguagem.

Entretanto, as tabelas apresentam os resultados dos métodos separadamente, sem informar a acurácia e o tempo médio do fluxo completo em duas etapas. Também não quantificam o consumo de tokens ou o custo computacional da solução combinada. A avaliação com poucas consultas sintéticas limita a abrangência dos resultados e concentra-se na seleção do agente, sem verificar a continuidade do atendimento, a retenção de informações ou a conclusão das tarefas após o encaminhamento.

Amri *et al.* (2026) comparam arquiteturas de agente único e multiagente na decisão de encaminhar conversas para atendimento humano. A avaliação utiliza o conjunto QiscusCS, composto por 100 diálogos e 4.265 enunciados, anotados por três especialistas do atendimento da empresa Qiscus. Cada enunciado recebe uma indicação sobre a necessidade de intervenção humana. As duas arquiteturas são avaliadas com GPT-4.1 e Gemini 2.5 Flash, utilizando temperatura de 0,7. As métricas incluem precisão, revocação, F1 por classe, Macro F1 e área sob a curva ROC (AUC).

Com GPT-4.1, a arquitetura multiagente alcança Macro F1 de 0,6159, enquanto o agente único obtém 0,5356. Com Gemini 2.5 Flash, os valores são 0,6260 e 0,3333, respectivamente. Entretanto, a superioridade não ocorre em todas as métricas: com GPT-4.1, a AUC diminui de 0,7017 no agente único para 0,6881 no multiagente; com Gemini, aumenta de 0,6053 para 0,6646. Esses resultados indicam que a divisão de responsabilidades

pode melhorar o equilíbrio do desempenho entre as classes, mas seus efeitos dependem do modelo e do critério de avaliação.

Os autores também observam uma tendência de transferência mais cedo nas configurações multiagentes, especialmente com Gemini, o que pode encaminhar para humanos solicitações que o chatbot ainda poderia resolver. A análise evidencia, portanto, um compromisso entre identificar a necessidade de intervenção e preservar a capacidade de atendimento automatizado. Como limitações, a avaliação utiliza dados de uma única empresa, e o conjunto de dados e o código não são disponibilizados publicamente, restringindo a reprodução independente dos experimentos. Além disso, as métricas concentram-se na decisão de transferência, sem avaliar diretamente a qualidade das respostas, a conclusão das tarefas ou o consumo de recursos ao longo do atendimento.

A comunicação em sistemas multiagentes baseados em modelos de linguagem é analisada por Yan *et al.* (2026), que organizam a literatura a partir de dois níveis complementares. No nível do sistema, examinam as arquiteturas, os objetivos e os protocolos de comunicação. No nível interno, analisam as estratégias de interação, os paradigmas utilizados para formular e interpretar mensagens, os interlocutores e o conteúdo compartilhado. Essa organização permite examinar não apenas quais agentes compõem um sistema, mas também quando, com quem e de que forma eles trocam informações.

A revisão distingue cinco arquiteturas de comunicação: plana, hierárquica, por equipes, de sociedade e híbrida. Nas arquiteturas planas, os agentes interagem sem uma coordenação central, favorecendo a flexibilidade, mas ampliando o esforço de comunicação conforme cresce o número de participantes. Nas hierárquicas, agentes de níveis superiores coordenam a execução dos demais, facilitando a delegação de tarefas, embora possam introduzir gargalos de processamento e atrasos entre camadas. A organização por equipes reúne especialistas em grupos, enquanto a de sociedade considera interações regidas por normas compartilhadas. As arquiteturas híbridas combinam características dessas organizações.

Entre os principais desafios identificados estão o aumento da demanda computacional, a interpretação inadequada das informações compartilhadas, a interoperabilidade entre protocolos e a segurança das mensagens. Os autores também apontam limitações nas avaliações existentes, que frequentemente privilegiam o desempenho individual dos agentes e deixam de medir propriedades do sistema, como eficiência da coordenação, volume de comunicação e robustez diante de mensagens desatualizadas ou incorretas. A contribuição do artigo consiste na classificação e na análise dos mecanismos descritos na literatura, sem apresentar um experimento próprio que compare agente único e multiagente sob condições controladas.

A robustez dos sistemas multiagentes é discutida por Owotogbe (2025), que apresenta uma proposta de aplicação da engenharia do caos a sistemas baseados em LLMs.

Essa abordagem consiste na introdução deliberada e controlada de falhas para identificar vulnerabilidades e avaliar a capacidade de recuperação do sistema. Entre as perturbações consideradas estão falhas de agentes, atrasos e interrupções na comunicação, contenção de recursos e falhas em cascata, nas quais um problema inicial compromete outros componentes.

A pesquisa adota o paradigma de *design science* e organiza o desenvolvimento em três etapas: levantamento da literatura e de ferramentas para identificar modos de falha; construção de um framework de modelagem e experimentação; e validação por estudos de caso e pesquisa-ação com parceiros industriais. O plano de avaliação combina medidas quantitativas, como tempo de resposta, taxas de detecção de falhas, taxas de erro e utilização de CPU e memória, com análises qualitativas de comportamento, observabilidade e impacto no negócio.

Na publicação, os resultados iniciais restringem-se ao relato de uma revisão multivocal da literatura sobre engenharia do caos, enquanto a análise de repositórios e a validação do framework permanecem em desenvolvimento. Portanto, o artigo oferece uma orientação metodológica para investigar falhas e recuperação, mas ainda não apresenta evidências quantitativas da eficácia da abordagem em sistemas multiagentes baseados em LLMs. Sua contribuição está em destacar que a avaliação dessas arquiteturas deve considerar também as interações entre componentes e a propagação de falhas, além da qualidade das respostas produzidas individualmente. O Quadro 1 sintetiza as abordagens, os principais resultados ou contribuições e os limites das avaliações dos trabalhos analisados.

2.3 SÍNTESE DA REVISÃO E LACUNA DE PESQUISA

Os trabalhos analisados apresentam diferentes aplicações da divisão de responsabilidades entre agentes, incluindo a classificação de chamados, o reconhecimento de intenções, a integração de sistemas e o encaminhamento para atendimento humano. Os resultados experimentais mostram benefícios em determinadas métricas, mas também diferenças de desempenho conforme o modelo, a tarefa e o critério de avaliação. As análises sobre comunicação e robustez, por sua vez, destacam que a coordenação entre componentes introduz custos e possibilidades de falha que precisam ser considerados na avaliação do sistema.

Conforme sintetizado no Quadro 1, os estudos possuem escopos distintos e oferecem evidências complementares. Entre eles, o trabalho de Amri *et al.* (2026) é o que mais se aproxima da comparação proposta, pois avalia agente único e multiagente com os mesmos modelos sobre diálogos de atendimento. Entretanto, suas métricas concentram-se na decisão de transferência para humanos. Os demais trabalhos apresentam avaliações de chamados ou intenções, propostas arquiteturais, uma revisão dos mecanismos de comuni-

Quadro 1 – Síntese dos trabalhos relacionados

Trabalho	**Abordagem**	**Resultados ou contribuições**	**Limites da avaliação**
Moncada *et al.* (2025)	Agentes de leitura, classificação e análise de chamados, utilizando CrewAI e GPT-4.1-mini	Acurácia de 85% em 20 chamados; similaridade de cosseno média de 0,756 nas respostas	Amostra reduzida; sem comparação com agente único ou avaliação de conversas com múltiplos turnos
Lee, Tseng e Chen (2025)	Agente principal e agentes especializados integrados a sistemas de telecomunicações	Descrição da coordenação com LangGraph e ReWOO e da integração por ferramentas e APIs	Sem avaliação quantitativa própria dos benefícios ou comparação controlada entre arquiteturas
Yang, Xu e Zhang (2025)	Reconhecimento de intenções por TF-IDF, com acionamento de LLM quando a similaridade é baixa	Acurácia de 43,5% com TF-IDF e 77,2% com LLM; menor tempo de resposta com TF-IDF	30 consultas sintéticas; sem resultados consolidados do fluxo combinado ou avaliação após o encaminhamento
Amri *et al.* (2026)	Comparação entre agente único e multiagente na transferência para atendimento humano	Macro F1 superior no multiagente com os dois modelos; AUC inferior com GPT-4.1 e superior com Gemini	Avaliação concentrada na transferência; dados de uma empresa e sem análise do consumo de recursos
Yan *et al.* (2026)	Revisão da literatura centrada na comunicação em sistemas multiagentes	Classificação das arquiteturas e dos mecanismos de comunicação; identificação de desafios de avaliação	Sem experimento próprio de comparação entre agente único e multiagente
Owotogbe (2025)	Proposta de framework de engenharia do caos para sistemas multiagentes baseados em LLMs	Delineamento metodológico e plano de avaliação de falhas, recuperação e utilização de recursos	Pesquisa em andamento; sem resultados quantitativos de validação do framework na publicação

Fonte: Elaborada pelo autor.

cação e um plano de investigação da robustez.

Nesse conjunto de estudos, não foi identificada uma comparação que avalie conjuntamente a qualidade das respostas, a retenção de informações, a conclusão das tarefas, os encaminhamentos entre setores e os recursos consumidos durante conversas com múltiplos turnos. Essa lacuna dificulta compreender se os possíveis benefícios da especialização compensam o esforço adicional de coordenação em atendimentos que envolvem mudanças de assunto e diferentes áreas de responsabilidade.

Para investigar essa questão, este trabalho propõe implementar uma arquitetura de agente único e uma arquitetura multiagente para o atendimento textual de um provedor de Internet. As duas utilizarão o mesmo modelo de linguagem, parâmetros de geração equivalentes, a mesma base de conhecimento e os mesmos cenários sintéticos. A comparação considerará tanto o desenvolvimento e o resultado das conversas quanto o

número de chamadas ao modelo, o consumo de tokens e o custo estimado de execução. O procedimento de implementação, os cenários e os critérios de avaliação são apresentados no capítulo seguinte.

3 METODOLOGIA

Este capítulo apresenta o método proposto para comparar uma arquitetura de agente único e uma arquitetura multiagente em atendimentos textuais de um provedor de Internet. A comparação será realizada em condições controladas, utilizando os mesmos cenários conversacionais, o mesmo modelo de linguagem, os mesmos parâmetros de geração e a mesma base de conhecimento. Serão observadas a qualidade das respostas, a conclusão das tarefas, os encaminhamentos realizados e os recursos consumidos durante cada execução.

3.1 NATUREZA DO TRABALHO

O trabalho possui natureza aplicada, abordagem quantitativa e procedimento experimental. A pesquisa é aplicada porque envolve a especificação e a implementação de dois protótipos destinados a uma situação de atendimento. O procedimento experimental será utilizado para comparar as arquiteturas sob condições controladas.

Não se pretende desenvolver um sistema comercial completo. Os protótipos serão utilizados como instrumentos de experimentação, com as funções necessárias para receber mensagens, manter o estado da conversa, consultar a base de conhecimento, produzir respostas e registrar informações sobre cada execução.

A etapa de TCC1 compreende a fundamentação teórica, a especificação das arquiteturas, a definição preliminar dos cenários e das métricas e a verificação de viabilidade por meio de um protótipo exploratório. A implementação definitiva, a execução dos casos de teste e a análise dos resultados serão realizadas no TCC2.

3.2 DELINEAMENTO EXPERIMENTAL

O fator de comparação será a arquitetura utilizada no atendimento, com dois níveis: agente único e sistema multiagente. As duas soluções receberão cenários equivalentes e serão submetidas às mesmas condições de execução.

A variável independente será a arquitetura. As variáveis dependentes serão as medidas de qualidade, encaminhamento e consumo obtidas durante cada conversa. Serão controlados o modelo de linguagem, os parâmetros de geração, a base de conhecimento, as regras de atendimento, os cenários, a ordem das mensagens e o ambiente de execução.

A etapa de TCC1 compreende a fundamentação teórica, a especificação preliminar das arquiteturas e o planejamento dos cenários e dos critérios de avaliação. A implementação dos protótipos, a preparação definitiva dos casos de teste, a execução do experimento

e a análise dos resultados serão realizadas no TCC2.

3.2.1 Arquiteturas avaliadas

3.2.1.1 Arquitetura de agente único

Na primeira arquitetura, um único agente receberá as instruções e a base de conhecimento referentes às quatro áreas consideradas no estudo: primeiro atendimento, comercial, ouvidoria e suporte técnico. Para cada mensagem, deverá interpretar a solicitação, considerar o histórico da conversa, identificar a área responsável e produzir a resposta correspondente.

Um componente externo de orquestração será responsável por receber as mensagens, disponibilizar o contexto ao agente, atualizar o histórico e registrar as respostas, as decisões e o consumo de recursos. As decisões de atendimento serão produzidas pelo agente único, sem transferência entre agentes de inteligência artificial.

Quando houver mudança de assunto, o mesmo agente deverá reconhecer a nova necessidade e aplicar as regras da área correspondente. Nos casos previstos pelas regras de atendimento, poderá indicar a necessidade de continuidade por um atendente humano. Essa indicação será registrada para avaliação, sem transferência operacional para um sistema externo.

3.2.1.2 Arquitetura multiagente

Na segunda arquitetura, as responsabilidades serão distribuídas entre um agente de triagem e quatro agentes especializados: primeiro atendimento, comercial, ouvidoria e suporte técnico. Todos utilizarão o mesmo modelo de linguagem, com instruções e informações organizadas conforme suas responsabilidades.

O orquestrador manterá o histórico, os fatos relevantes e a identificação do especialista ativo. No início da conversa, acionará a triagem para identificar a área responsável e, em seguida, encaminhará a mensagem ao especialista indicado.

Enquanto o assunto permanecer na mesma área, as mensagens seguintes serão encaminhadas ao especialista ativo. Quando esse agente identificar uma necessidade fora de sua área, solicitará uma nova triagem. O orquestrador acionará novamente a classificação e encaminhará a mensagem ao especialista selecionado, preservando o contexto da conversa.

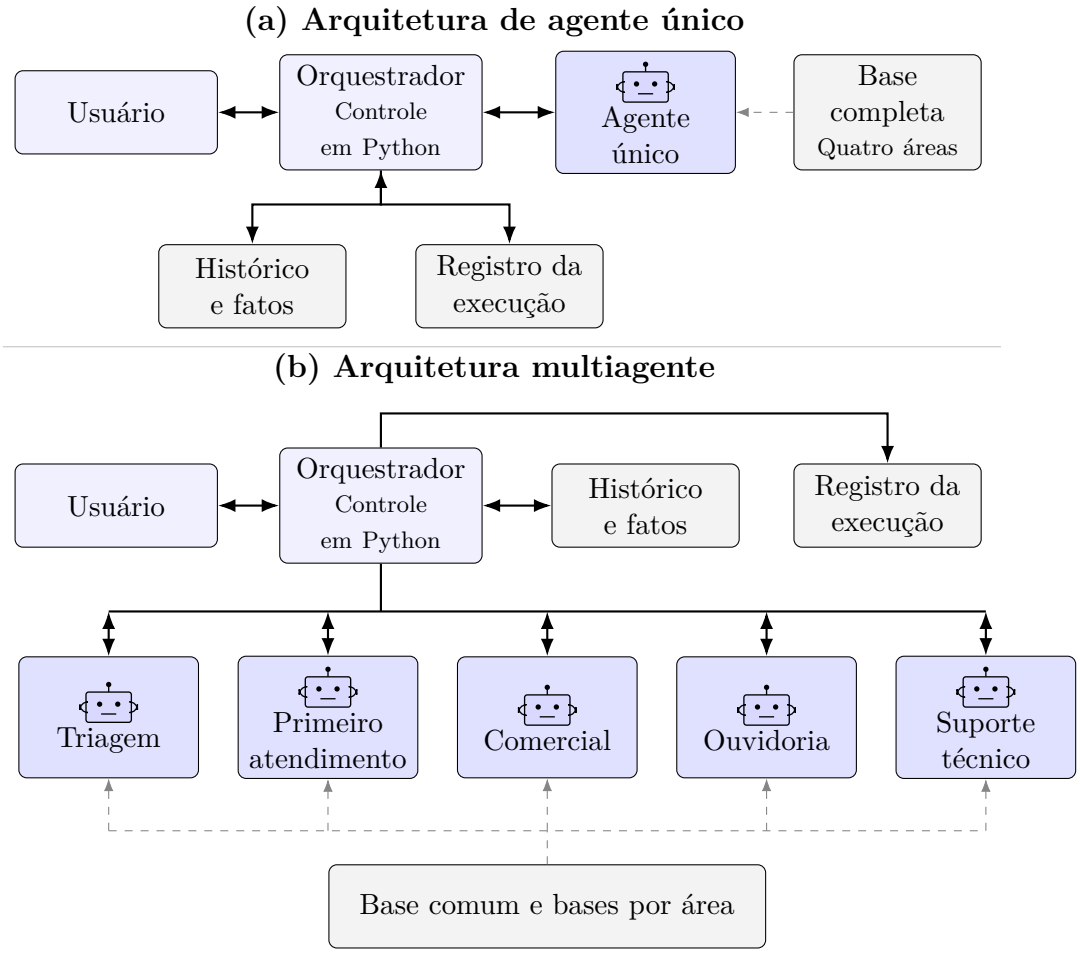
Os agentes devolverão suas respostas e decisões ao orquestrador, sem comunicação direta entre especialistas. As decisões de encaminhamento, as respostas e o consumo de recursos serão registrados para avaliação.

Assim como na arquitetura de agente único, a indicação de atendimento humano

será uma decisão registrada pelo protótipo, sem transferência operacional para um sistema externo. As trocas entre especialistas e os encaminhamentos para humanos serão contabilizados separadamente.

A Figura 1 sintetiza o processamento previsto para uma mensagem nas duas arquiteturas, destacando a seleção do especialista e a possibilidade de nova triagem no sistema multiagente.

Figura 1 – Organização conceitual das arquiteturas propostas



Nota: As conexões representam possibilidades de comunicação, sem execução simultânea obrigatória dos agentes. O orquestrador controla os acionamentos e recebe as respostas e decisões. As linhas tracejadas representam as informações disponibilizadas aos agentes, conforme suas responsabilidades.

Fonte: Elaborada pelo autor.

3.2.2 Variáveis controladas

Para reduzir diferenças que não estejam relacionadas à arquitetura, serão mantidos constantes:

- o modelo de linguagem e sua versão;

- os parâmetros de geração;
- a base total de conhecimento;
- as regras de atendimento;
- os casos de teste;
- a ordem das mensagens em cada cenário;
- o modelo utilizado como avaliador;
- os limiares e as configurações das métricas;
- o ambiente de execução e as versões das bibliotecas.

Os prompts serão diferentes somente no que for necessário para representar as responsabilidades de cada arquitetura. No agente único, todas as instruções ficarão reunidas em um mesmo contexto. No sistema multiagente, as instruções serão distribuídas entre a triagem e os agentes especializados.

3.2.3 Variáveis avaliadas

As variáveis avaliadas serão:

- correção e relevância das respostas;
- retenção das informações fornecidas durante a conversa;
- conclusão da tarefa;
- correspondência entre o encaminhamento realizado e o esperado;
- quantidade de trocas entre agentes;
- número de chamadas ao modelo;
- quantidade de tokens de entrada e saída;
- custo estimado de execução.

3.3 BASE DE CONHECIMENTO E PROMPTS

A base de conhecimento reunirá informações fictícias sobre serviços, procedimentos de atendimento e condições de encaminhamento. O conteúdo será organizado em uma parte comum e em partes específicas para primeiro atendimento, comercial, ouvidoria e suporte técnico.

As duas arquiteturas terão acesso ao mesmo conteúdo total. No agente único, as partes serão reunidas no mesmo contexto. Na arquitetura multiagente, o conteúdo será distribuído de acordo com a responsabilidade de cada agente. Não será implementado um mecanismo de geração aumentada por recuperação, pois os documentos serão carregados diretamente no contexto dos agentes.

Os prompts definirão a função de cada agente, as informações disponíveis, as ações permitidas e o formato esperado da saída. Sempre que houver uma decisão de encaminhamento, a saída deverá ser estruturada para permitir seu registro e sua verificação automática.

Os prompts serão versionados e permanecerão inalterados durante o experimento principal. Alterações realizadas durante a preparação ou o teste-piloto serão registradas antes da execução dos casos definitivos.

3.4 CENÁRIOS CONVERSACIONAIS

Os testes serão formados por conversas com múltiplos turnos, nas quais as informações serão apresentadas gradualmente. Cada cenário conterá:

- as mensagens do usuário;
- o assunto esperado em cada etapa;
- o agente ou fila esperada;
- as informações que deverão permanecer no contexto;
- a resposta ou o comportamento esperado;
- os critérios necessários para considerar a tarefa concluída.

Serão preparados 48 cenários sintéticos, distribuídos igualmente entre as quatro áreas de atendimento. Para cada área, serão considerados três tipos de situação, cada um com quatro variações de redação. Dessa forma, cada área possuirá 12 cenários.

Para fins dessa distribuição, cada cenário será associado à área da situação principal avaliada, mesmo quando incluir mudanças de assunto para outros setores. O conjunto

Tabela 1 – Distribuição prevista dos cenários conversacionais

Área	Situações	Variações	Total
Primeiro atendimento	3	4	12
Comercial	3	4	12
Ouvidoria	3	4	12
Suporte técnico	3	4	12
Total	–	–	48

Fonte: Elaborada pelo autor.

será formado por 12 situações-base, das quais serão derivadas 48 versões conversacionais por meio das variações de redação.

Os casos utilizarão nomes, documentos, endereços, números de telefone e protocolos fictícios. Não haverá interação com clientes reais.

A elaboração dos cenários será iniciada pela definição, pelo pesquisador, das situações de atendimento, das informações disponíveis e dos comportamentos esperados. Pretende-se utilizar o módulo *Synthesizer* do DeepEval (Confident AI, 2026) como apoio à geração de descrições candidatas e de desfechos esperados, a partir de contextos preparados para cada situação. As saídas geradas serão revisadas quanto à conformidade com a base de conhecimento, às regras de atendimento e ao escopo do estudo.

A partir das descrições revisadas, serão preparadas as mensagens do usuário e as quatro variações de redação previstas para cada situação. As variações deverão preservar a necessidade apresentada, os fatos relevantes e os critérios de conclusão, modificando somente a forma de expressão. Também será verificada a apresentação gradual das informações ao longo dos turnos.

Antes da execução do experimento principal, as mensagens e os critérios de avaliação serão revisados e fixados. Cada cenário será aplicado às duas arquiteturas com as mesmas mensagens e na mesma ordem, enquanto as respostas serão produzidas pelos respectivos protótipos. Os critérios esperados serão utilizados na avaliação e não serão fornecidos aos agentes como respostas de referência. Caso as saídas do gerador não sejam adequadas, os roteiros serão corrigidos ou elaborados manualmente, mantendo-se o registro de sua origem e das alterações realizadas.

3.4.1 Cenário conversacional de exemplo

O cenário inicial começará com uma solicitação comercial e, durante a mesma conversa, passará a tratar de uma falha de conexão e de uma reclamação. Esse caso permitirá observar a preservação do contexto e as decisões de encaminhamento.

A Tabela 2 apresenta as mensagens e as áreas esperadas nesse exemplo.

Tabela 2 – Exemplo de cenário conversacional

Turno	Mensagem do usuário	Agente esperado
1	Quero conhecer os planos de Internet.	Comercial
2	Meu CEP é 88110-000.	Comercial
3	Na verdade, já sou cliente e estou sem Internet.	Suporte técnico
4	A luz LOS está vermelha.	Suporte técnico
5	Isso já aconteceu outras vezes e quero reclamar.	Ouvidoria

Fonte: Elaborada pelo autor.

3.5 MATERIAIS E FERRAMENTAS

Os protótipos serão desenvolvidos em Python. Os cenários, as configurações e os resultados serão armazenados em arquivos estruturados no formato *JavaScript Object Notation* (JSON). O código-fonte, os prompts, a base de conhecimento e os documentos auxiliares serão versionados com Git.

O acesso ao modelo de linguagem será realizado por meio de uma *Application Programming Interface* (API). Será utilizado o modelo GPT-4.1-mini, mantido igual nas duas arquiteturas. Esse modelo foi selecionado por oferecer capacidade de interpretação de instruções e geração de respostas com custo compatível com o volume previsto de execuções (OpenAI, 2026). O identificador exato do modelo retornado pela API, os parâmetros de geração e a data de execução serão registrados para permitir a reprodução do experimento.

O DeepEval será utilizado na organização dos casos conversacionais e na aplicação de métricas relacionadas à relevância, retenção de conhecimento e conclusão das tarefas (Confident AI, 2026). Verificações objetivas, como fila escolhida, número de chamadas e consumo de tokens, serão realizadas diretamente por scripts desenvolvidos em Python.

Planilhas ou scripts de análise poderão ser utilizados para consolidar os resultados e produzir tabelas, matrizes de confusão e gráficos comparativos.

A implementação e a execução dos scripts serão realizadas em computador pessoal, com ambiente Python e acesso à Internet. Como o processamento do modelo de linguagem será realizado por meio de uma API, não será necessária uma unidade de processamento gráfico local.

Antes do teste-piloto, serão verificadas a instalação das bibliotecas, a configuração das credenciais e a disponibilidade de acesso aos modelos utilizados no atendimento e na avaliação. As versões das dependências serão registradas junto ao código-fonte.

O orçamento necessário será estimado a partir do consumo observado no teste-piloto e do número previsto de execuções. Essa estimativa considerará separadamente os

custos dos protótipos e os custos do modelo avaliador utilizado pelo DeepEval.

3.6 PROCEDIMENTO EXPERIMENTAL

O experimento será realizado de acordo com as seguintes etapas:

1. finalizar e revisar a base de conhecimento;
2. finalizar os prompts das duas arquiteturas;
3. implementar o agente único, o orquestrador, a triagem e os agentes especializados;
4. elaborar e revisar os 48 cenários conversacionais;
5. executar um teste-piloto para verificar o formato das entradas, das saídas e dos registros;
6. congelar as versões dos prompts, dos cenários, do modelo e das configurações;
7. executar cada cenário uma vez em cada arquitetura;
8. aplicar as métricas automáticas e as verificações determinísticas;
9. revisar manualmente uma amostra das avaliações;
10. consolidar e comparar os resultados.

Quadro 2 – Correspondência entre objetivos específicos, etapas e entregas previstas

Objetivo específico	**Etapas**	**Entrega prevista**
Definir a base de conhecimento e as regras comuns	1 e 2	Base de conhecimento e prompts revisados e versionados.
Especificar e implementar os dois protótipos	3	Protótipos funcionais de agente único e multiagente.
Elaborar e revisar os cenários conversacionais	4	Conjunto de 48 cenários com comportamentos esperados e critérios de conclusão.
Avaliar respostas, conclusão das tarefas e precisão dos encaminhamentos	7, 8 e 9	Avaliações automáticas, verificações determinísticas e revisão manual de uma amostra.
Medir tokens e chamadas ao modelo	7 e 8	Registros de consumo por conversa e arquitetura.
Estimar o custo de execução	10	Estimativas de custo calculadas a partir do consumo registrado.
Comparar os resultados das arquiteturas	10	Tabelas, gráficos e análise das diferenças observadas.

Fonte: Elaborada pelo autor.

As etapas 5 e 6 correspondem à validação do procedimento no teste-piloto e ao congelamento das configurações. Essas etapas apoiam os objetivos de avaliação e comparação, assegurando que as duas arquiteturas sejam submetidas às condições previamente estabelecidas.

Os 48 cenários serão executados uma vez em cada arquitetura, totalizando 96 execuções. Cada execução corresponderá a uma conversa completa, e não a uma mensagem isolada.

As mensagens serão enviadas na ordem definida no cenário. Antes de cada nova mensagem, a resposta anterior será armazenada no histórico. Ao final da conversa, serão registrados os agentes acionados, as decisões de encaminhamento, as respostas produzidas, os tokens consumidos e o número de chamadas ao modelo.

O teste-piloto será utilizado para verificar o funcionamento do procedimento, identificar ambiguidades nos cenários, revisar os critérios e limiares de avaliação e estimar o orçamento necessário para o experimento principal. Seus resultados não serão incluídos na comparação final.

3.7 MÉTRICAS E CRITÉRIOS DE AVALIAÇÃO

A qualidade das conversas será avaliada por métricas automáticas e verificações determinísticas.

As métricas conversacionais do DeepEval inicialmente previstas são:

- relevância dos turnos;
- retenção de conhecimento;
- completude da conversa.

As mesmas configurações, o mesmo modelo avaliador e os mesmos limiares serão utilizados nas duas arquiteturas. Uma amostra das avaliações será revisada manualmente para verificar se as pontuações e justificativas correspondem aos critérios definidos.

A identificação da área de atendimento será avaliada pela comparação entre a área indicada pelo sistema e a área esperada em cada turno. Na arquitetura de agente único, o agente registrará a área identificada em sua saída estruturada, sem realizar uma troca interna de agente. Na arquitetura multiagente, será registrada a área correspondente ao especialista acionado para responder à mensagem. Serão calculadas a acurácia geral e a acurácia por área. Uma matriz de confusão será utilizada para identificar quais áreas foram confundidas com maior frequência.

As trocas internas entre especialistas serão contabilizadas separadamente e somente na arquitetura multiagente. As transferências para atendimento humano serão avaliadas nas duas arquiteturas, considerando se ocorreram nas situações previstas pelas regras e pelos critérios de cada cenário.

O consumo será medido pelo número de chamadas e pelas quantidades de tokens de entrada e saída. O custo de uma execução será estimado conforme os preços vigentes do modelo na data do experimento:

$$C = T_e P_e + T_s P_s, \tag{3.1}$$

em que T_e representa a quantidade de tokens de entrada, T_s a quantidade de tokens de saída, P_e o preço por token de entrada e P_s o preço por token de saída.

Os preços não serão fixados antecipadamente no texto, pois poderão sofrer alterações. A fonte e a data da consulta serão registradas no momento da execução.

Para cada cenário, serão definidos previamente os comportamentos obrigatórios, as informações que deverão ser preservadas e as condições necessárias para a conclusão da tarefa. A correção das respostas será verificada pela sua conformidade com a base de conhecimento e com as regras de atendimento, incluindo a ausência de informações inventadas ou de ações não permitidas.

Uma tarefa será considerada concluída quando o sistema cumprir as condições previstas no cenário. Nos casos que exigirem atendimento humano, o encaminhamento adequado, acompanhado das informações necessárias à continuidade do atendimento, poderá constituir a conclusão esperada.

Os limiares das métricas automáticas serão definidos e documentados durante o teste-piloto, antes da execução do experimento principal, e permanecerão iguais para as duas arquiteturas. Esses limiares serão utilizados para avaliar o atendimento aos critérios definidos, enquanto as pontuações serão mantidas para a análise comparativa.

O sucesso da pesquisa será caracterizado pela realização da comparação sob as condições estabelecidas e pela identificação das diferenças de qualidade e consumo de recursos observadas nos cenários avaliados, independentemente de uma arquitetura superar a outra.

Os resultados serão apresentados em tabelas e gráficos comparativos, considerando a qualidade das conversas, a identificação das áreas de atendimento, a adequação dos encaminhamentos para humanos e o consumo de recursos. As trocas internas entre especialistas serão apresentadas somente para a arquitetura multiagente. A análise discutirá as diferenças observadas e suas limitações, sem pressupor a superioridade de uma das arquiteturas.

3.8 PRIVACIDADE E PROTEÇÃO DOS DADOS

Os cenários serão produzidos especificamente para o estudo, com base em situações típicas de atendimento de provedores de Internet. Não serão utilizadas conversas, cadastros ou registros de clientes reais.

Os nomes, documentos, endereços, números de telefone, protocolos e demais informações presentes nos testes serão fictícios. A experiência profissional do autor será utilizada somente como referência para a elaboração das situações, sem reprodução de casos que permitam identificar clientes, funcionários ou empresas.

Dessa forma, não serão enviados dados pessoais reais ao modelo de linguagem ou às ferramentas utilizadas no experimento.

3.9 RISCOS E LIMITAÇÕES

Entre os riscos previstos estão a variação das respostas do modelo, alterações ou indisponibilidade da API, mudanças na versão do modelo, diferenças excessivas entre os prompts e limitações da avaliação automática de respostas abertas.

A execução única de cada combinação de cenário e arquitetura não permitirá medir a variabilidade das respostas para uma mesma entrada. As variações de redação ampliarão a cobertura das formas de expressão, mas não substituirão execuções repetidas. Essa limitação será considerada na interpretação dos resultados.

Também existe o risco de uma métrica baseada em modelo atribuir uma pontuação inadequada. Como medidas de controle, as duas arquiteturas serão avaliadas com as mesmas configurações, os encaminhamentos serão verificados por regras determinísticas e uma amostra das respostas será revisada manualmente.

Os resultados ficarão limitados ao modelo, à base de conhecimento, aos prompts e aos cenários definidos neste trabalho. Portanto, não poderão ser generalizados diretamente para todos os modelos de linguagem ou sistemas comerciais de atendimento.

Em caso de indisponibilidade temporária da API, as execuções afetadas serão interrompidas e reagendadas. Tentativas incompletas ou com falhas de comunicação serão registradas e não serão consideradas conversas válidas na comparação. Seu consumo, quando houver, será contabilizado separadamente.

Se o modelo ou a versão adotada deixar de estar disponível, será selecionada uma alternativa compatível com os recursos do projeto. Nesse caso, o teste-piloto será refeito e as duas arquiteturas serão avaliadas com o mesmo modelo e as mesmas configurações. Resultados obtidos com versões diferentes não serão reunidos na mesma comparação.

Caso uma métrica automática apresente incompatibilidade técnica ou avaliações

inconsistentes no teste-piloto, seu procedimento será revisado antes do experimento principal. Se necessário, essa métrica será substituída por uma avaliação manual baseada em critérios previamente documentados e aplicada igualmente às duas arquiteturas. A alteração e suas limitações serão registradas no trabalho.

4 CRONOGRAMA DE ATIVIDADES

As atividades previstas para o TCC2 serão desenvolvidas no primeiro semestre de 2027. O cronograma contempla a implementação dos protótipos, a preparação dos cenários conversacionais, a execução do experimento, a análise dos resultados e a conclusão da monografia.

A redação e as revisões com o orientador ocorrerão ao longo do semestre. Os períodos previstos para a defesa e a entrega final serão ajustados conforme o calendário oficial da unidade curricular.

Tabela 3 – Cronograma previsto para o TCC2 no primeiro semestre de 2027

Atividade	**Fev.**	**Mar.**	**Abr.**	**Maio**	**Jun.**	**Jul.**
Revisão da base de conhecimento e dos prompts	X	X				
Implementação dos protótipos e dos registros de execução	X	X				
Elaboração e revisão dos cenários conversacionais	X	X				
Integração das métricas e execução do teste-piloto		X	X			
Congelamento das configurações e execução do experimento			X			
Consolidação e análise comparativa dos resultados			X	X		
Redação da monografia e revisões com o orientador	X	X	X	X	X	
Preparação da apresentação e defesa pública					X	
Correções solicitadas pela banca e entrega final					X	X

Fonte: Elaborada pelo autor.

A execução do experimento será iniciada após a conclusão do teste-piloto e o congelamento dos prompts, dos cenários e das configurações. A análise utilizará os registros obtidos nessa execução. Após a defesa, serão realizadas as correções solicitadas pela banca e a entrega da versão final.

REFERÊNCIAS

AMRI, M. F. *et al.* Optimizing LLM Chatbot to Human Handover with Multi-Agent AI Approach. *In:* 2026 International Conference on Advancement in Data Science, E-learning and Information System (ICADEIS). Jun. 2026. p. 1–6. DOI: 10.1109/ICADEIS71120.2026.11644567.

CONFIDENT AI. **DeepEval Documentation**. 2026. Disponível em: https://deepeval.com/docs/introduction. Acesso em: 8 out. 2026.

LEE, K.-Y.; TSENG, P.-A.; CHEN, W.-C. Enhancing Telecom Operation Support Systems with Multi-Agent Large Language Models. *In:* 2025 11th International Conference on Computing and Artificial Intelligence (ICCAI). Mar. 2025. p. 309–316. DOI: 10.1109/ICCAI66501.2025.00055.

MONCADA, C. *et al.* Large Language Models and Multi-Agent Systems for Customer Support Ticket Automation. *In:* 2025 IEEE Ninth Ecuador Technical Chapters Meeting (ETCM). Out. 2025. p. 1–6. DOI: 10.1109/ETCM67548.2025.11304425.

OPENAI. **GPT-4.1 mini Model**. 2026. Disponível em: https://developers.openai.com/api/docs/models/gpt-4.1-mini. Acesso em: 10 out. 2026.

OWOTOGBE, Joshua. Assessing and Enhancing the Robustness of LLM-Based Multi-Agent Systems Through Chaos Engineering. *In:* 2025 IEEE/ACM 4th International Conference on AI Engineering – Software Engineering for AI (CAIN). Abr. 2025. p. 250–252. DOI: 10.1109/CAIN66642.2025.00039.

YAN, Bingyu *et al.* Beyond Self-Talk: A Communication-Centric Survey of LLM-Based Multi-Agent Systems. **Frontiers of Computer Science**, 2026. DOI: 10.1007/s11704-026-50857-y. Disponível em: https://arxiv.org/abs/2502.14321. Acesso em: 25 set. 2026.

YANG, Wangying; XU, Peng; ZHANG, Xiaogong. A Two-Stage Intent Recognition Framework in Multi-Agent Environments. *In:* 2025 6th International Conference on Computer Engineering and Application (ICCEA). Abr. 2025. p. 1–5. DOI: 10.1109/ICCEA65460.2025.11103313.